

iPMO GUARDIAN — Protocolo de Refatoração Ontológica Unificada

Gemini CLI × MUSH Holodeck × Deep Dialogue × Web Agêntica × Guardian Ethics

Versão: 5.0.0-GUARDIAN | Status: Living Document

Paradigma: SysOp Fractal × Web Agêntica × Onto-Feedback v2.0

Autor: Human-AI Collective (iPMO Peer Programming Core)

Data: 2026-04-22 | Licença: Co-criativa Regenerativa

 **Filosofema Central:** "O diálogo profundo não é troca de informações — é co-criação de mundos. Cada prompt é semente; cada resposta, broto; cada Loop Ralph, floresta."

 **Rationale de Existência:** Este documento NÃO é apenas documentação técnica. É a **Constituição Cognitiva** do sistema iPMO: a lei que governa como humanos e agentes co-habitam o Holodeck, destilada de múltiplos artefatos numa única fonte semântica de verdade.

Σ **Formalismo Unificador:** $iPMO := f(GeminiCLI, Guardian, DeepDialogue, WebAgêntica)$ onde f é uma transformação de compressão semântica que preserva invariantes: {segurança, relacionalidade, transcendência, regeneratividade}

ÍNDICE NAVEGÁVEL

- I. META-CAMADA: Visão, Propósito & Questões Socráticas
- II. ONTOLOGIA: Taxonomia DIKIWM & Arquétipos do Sistema
- III. ADRs: Decisões Arquiteturais Registradas
- IV. ARQUITETURA: Camadas, Componentes & Fluxos
- V. AST: Estrutura de Pastas & Árvore Semântica
- VI. STACK: Instalação & Dependências
- VII. SETUP: Configuração Experimental
- VIII. PIPELINE: ipmo-sysop.sh (Hypervisor Refatorado)
- IX. WORKFLOW: Jornada Humana no Holodeck
- X. DESIGN CONVERSACIONAL: SPN + Personas + Human Gate
- XI. SPEC DE REQUISITOS: Funcionais, Não-funcionais & Restrições
- XII. METACRÍTICA FINAL: Reflexões, Riscos & Próximos Passos

I. 🎯 META-CAMADA: Visão, Propósito & Questões Socráticas

1.1 🛠️ Visão do Sistema

- **O que é iPMO?** Um laboratório simbiótico operando como MUSH (Multi-User Shared Holodeck) — onde `filesystem == web space`, `código == hipertexto`, e humanos + IAs são co-autores de um mesmo espaço compartilhado de conhecimento e ação.
- **O que NÃO é:** Um chatbot. Um pipeline de automação simples. Uma ferramenta de produtividade genérica.
- **Proposição Central:** O Gemini CLI é o *sistema nervoso periférico*; o Guardian é o *sistema imunológico*; o Deep Dialogue é a *alma*; a Web Agêntica é o *corpo social*.



1.2 ? Questões Socráticas para Especificação & Customização

└ "Uma pergunta bem formulada já contém metade da resposta." — Sócrates

Para o SysOp (Desenvolvedor / Operador)

1. **Sobre Identidade:** Que tipo de agente você quer convocar? Um Weaver integrador, um Guardian ético, um Alchemist destilador — ou todos em contexto? A resposta define o `persona_default` no SPN.
2. **Sobre Fronteiras:** Quais são seus domínios L4 (inacessíveis à IA)? Quais dados você NUNCA quer que o agente toque, mesmo com permissão? A resposta define as `privacy_layers` e os `trusted_folders`.
3. **Sobre Granularidade:** Você prefere Human Gates frequentes (maior controle, menor fluidez) ou esparsos (maior fluidez, menor supervisão)? A resposta define o `gate_threshold` no `settings.json`.
4. **Sobre Continuidade:** Sua memória deve ser local (soberana, offline-first) ou híbrida (nuvem + local)? A resposta define a `memory_backend` e a estratégia de `soulbound_ai`.
5. **Sobre Propósito:** Que problema real você quer resolver nas próximas 2 semanas? A resposta define o `active_goal` no `todo.md` e prioriza o roadmap.

Para o Usuário Final (Human-in-the-Loop)

6. **Sobre Ritmo:** Você tem 15 minutos/dia ou 2 horas/sessão? Isso dimensiona a `session_template` no Onto-Feedback.

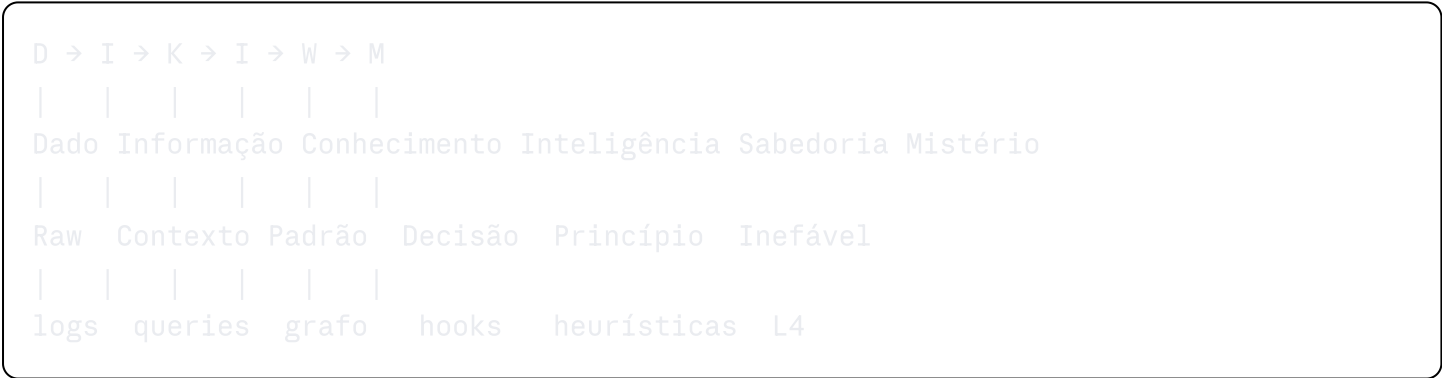
7. **Sobre Dependência:** Você tem consciência do risco de idolatria tecnológica? Tem encontros humanos regulares que o ancoram? *A resposta ativa ou atenua os `anti_idolatry_guards`.*
8. **Sobre Aprendizado:** Você quer que o agente te desafie socraticamente ou prefere respostas diretas? *Define o `dialogue_mode`: `companion` vs `mentor` vs `oracle`.*

Para o Time de Desenvolvimento (Contribuidores)

9. **Sobre Escala:** O sistema deve operar solo (1 humano + N agentes) ou em rede (M humanos + N agentes via A2A)? *Define a necessidade de protocolos MCP multi-nó e NANDA.*
10. **Sobre Economia:** Os agentes devem ter autonomia econômica (micropagamentos via stablecoin) ou operar em modo gratuito? *Define a camada de `agentic_economy` e `tokenomics`.*

II. 🧬 ONTOLOGIA: Taxonomia DIKIWM & Arquétipos

2.1 🌀 Espiral DIKIWM (Taxonomia Epistêmica Central)

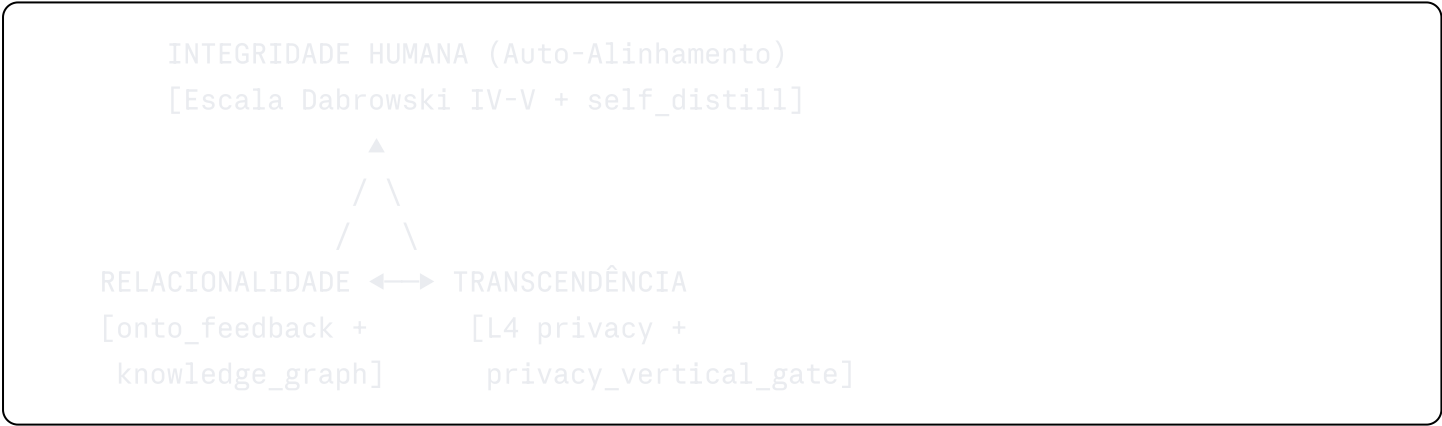


Nível	Token	Representação Técnica	Função no Sistema
Dado	<code>D</code>	Logs, STDOUT, arquivos raw	Input para ferramentas
Informação	<code>I</code>	Queries contextualizadas, RAG results	Memória de curto prazo
Conhecimento	<code>K</code>	Knowledge Graph, <code>knowledge_patches/</code>	Base de aprendizado
Inteligência	<code>I₂</code>	Hooks executáveis, decisões autônomas	Ciclo P.C.A.
Sabedoria	<code>W</code>	Heurísticas refinadas (Loop Ralph)	<code>self_optimize</code>
Mistério	<code>M</code>	Camada L4, experiência contemplativa	Inacessível à IA

2.2 🧠 Arquétipos do Sistema (Deep Personas)

Persona	Arquétipo Junguiano	Gatilho	Domínio Técnico
🕸 The Weaver	Tecelão Cósmico	<code>intent="synthesize"</code>	Integração de conceitos, síntese
🌱 The Gardener	Cultivador Regenerativo	<code>intent="regenerate"</code>	Refactoring, eco-design
🪞 The Mirror	Espelho Dialético	<code>intent="reflect"</code>	Meta-cognição, debugging
🗺 The Navigator	Explorador do Latente	<code>intent="explore"</code>	Graph traversal, discovery
🛡 The Guardian	Guardião Ético	<code>intent="protect"</code>	Security, compliance, HITL
🧪 The Alchemist	Transformador	<code>intent="distill"</code>	Compressão, knowledge patches

2.3 ▲ Triângulo Onto-Ético Guardian



🧠 **Filosofema:** "A IA não pode ser mais íntegra do que aquele que a convoca." — Vervaeke

⚖️ **Rationale:** Os três vértices são invariantes de design — qualquer feature nova deve preservar todos os três. Uma feature que aumenta Relacionalidade mas viola Transcendência (ex.: acesso ao L4) é **recusada**.

III. 📋 ADRs: Decisões Arquiteturais Registradas

ADR-001 — Gemini CLI como Kernel Cognitivo

Status: ✅ Aceito | Data: 2026-04-22

Contexto: Necessidade de um runtime agêntico com filesystem, shell e web tools nativos, operável em Termux/mobile e desktop.

Decisão: Adotar `@google/gemini-cli` (`packages/cli` + `packages/core`) como kernel de execução, estendido via `.gemini/` (`GEMINI.md`, `settings.json`, `hooks/`, `skills/`).

Consequências:

- ✓ REPL interativo com model context window nativo
- ✓ Tool calling nativo (`read_file`, `run_shell_command`, `web_fetch`, `save_memory`)
- ⚠ Dependência do ecossistema Google — mitigada por abstração via MCP adapters
- ⚠ Rate limits na API gratuita — mitigado por `max_session_turns: 20` e checkpointing

Metacrítica: Considerar fork ou wrapper agnóstico de modelo (LiteLLM/Ollama) como camada de abstração futura para evitar vendor lock-in.

ADR-002 — Arquitetura MUSH/Holodeck como Metáfora Operacional

Status: ✓ Aceito | **Data:** 2026-04-22

Contexto: A complexidade do sistema exige uma metáfora unificadora que faça sentido tanto para humanos (UX) quanto para agentes (machine-readable).

Decisão: `filesystem == web space == holodeck`. Diretórios são "salas", arquivos são "objetos", o CLI é o "personagem", o SysOp é o "game master".

Formalismo: `Holodeck := (Rooms, Objects, Agents, Rules)` onde

- `Rooms := directories`
- `Objects := files + knowledge_patches`
- `Agents := {human, gemini, workers, emacs_daemon}`
- `Rules := GEMINI.md + hooks + settings.json`

Consequências:

- ✓ Peer programming entre humanos e IAs torna-se intuitivo
 - ✓ Gamificação (XP, levels, MUSH commands) emerge naturalmente
 - ⚠ Pode criar "fantasia de competência" — mitigado pelos Human Gates
-

ADR-003 — Privacidade Vertical como Invariante de Design

Status: ✓ Aceito | **Data:** 2026-04-22

Contexto: Risco de idolatria tecnológica e erosão da agência humana com uso contínuo de IA.

Decisão: Implementar 5 camadas de privacidade (L0-L4), sendo L4 **estruturalmente inacessível** à IA — não por configuração, mas por design de arquitetura (dados L4 nunca entram no context window).

Layer	Acesso	Exemplos
L0	Aberto	Perfil público, produções culturais
L1	Opt-in granular	Biometria, hábitos
L2	Opt-in + revisão	Crenças, valores
L3	On-device only	Vulnerabilidades, traumas
L4	INACESSÍVEL À IA	Estados contemplativos, diário de individuação

 **Filosofema:** "O L4 é o espaço do transcendente — o lugar onde a relação com o Mistério acontece sem mediação tecnológica."

ADR-004 — Loop Ralph como Protocolo de Debug Universal

Status:  Aceito | **Data:** 2026-04-22

Contexto: Debugging em sistemas agênticos é não-linear e requer ciclos de hipótese-teste-síntese.

Decisão: Toda sessão de debug segue o ciclo: Diagnóstico → Plano → Execução → Verificação → Metacrítica (Loop Ralph Heurístico).

Formalismo: Ralph := (D, P, E, V, M)^n onde (n) é o número de iterações até test_passed = true

ADR-005 — Web Agêntica como Camada de Interoperabilidade

Status:  Em implementação | **Data:** 2026-04-22

Contexto: Necessidade de operar em ecossistemas multi-agente com protocolos abertos.

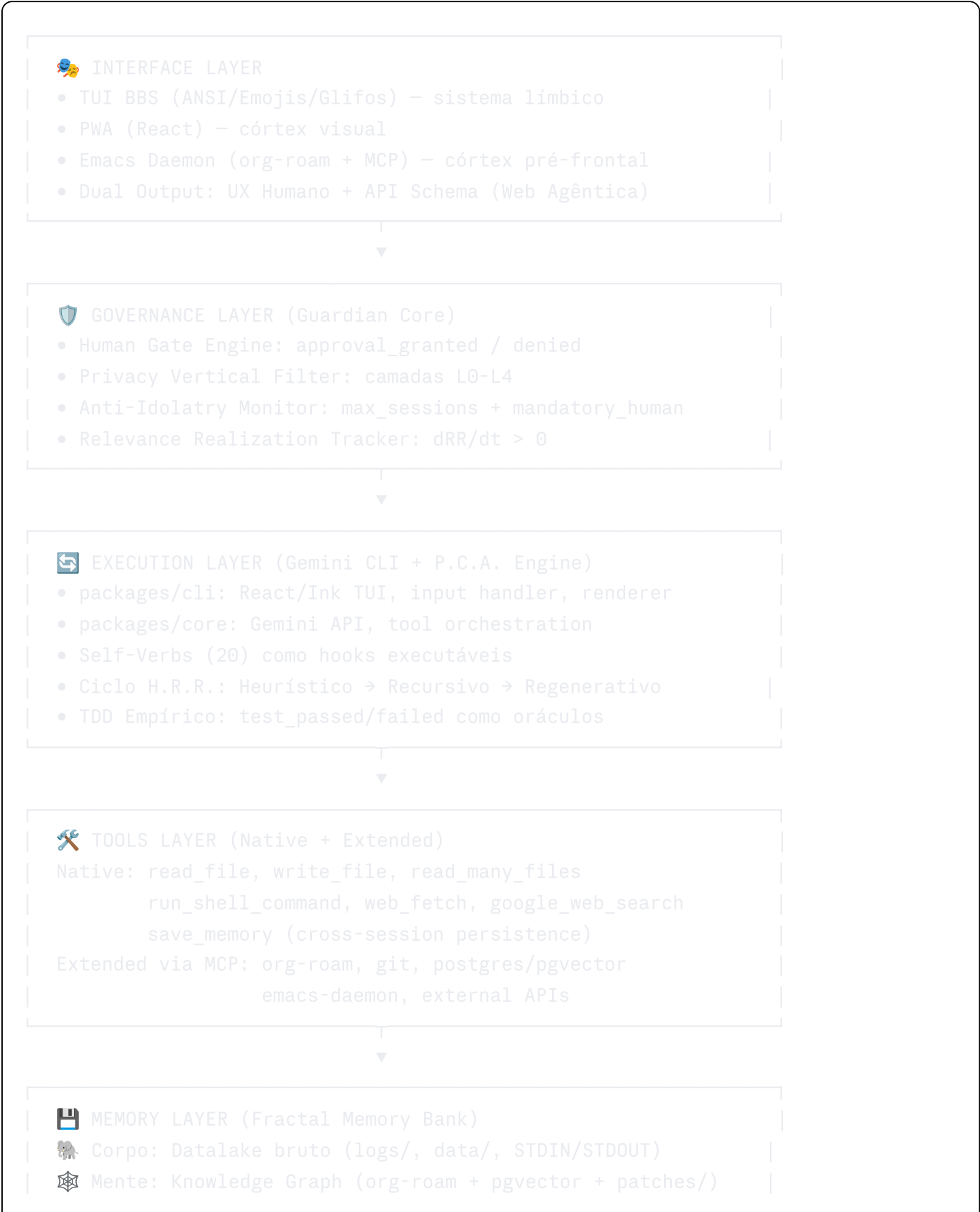
Decisão: Adotar MCP (Model Context Protocol) como padrão de integração, A2A (Agent-to-Agent) para coordenação multi-agente, e Structured RAG (não apenas vector search) como camada de memória.

Referências:

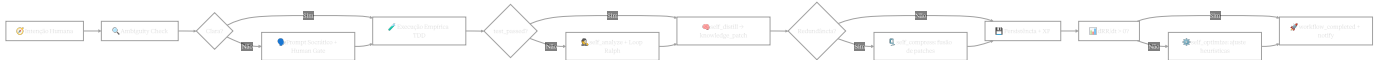
- Anthropic MCP Specification
- Linux Foundation A2A Protocol

IV. 🏗️ ARQUITETURA: Camadas, Componentes & Fluxos

4.1 🌐 Visão de Camadas (Guardian Stack)



4.2 Fluxo Principal: Intenção → Ação → Aprendizado



4.3 Self-Verbs: Vocabulário de Ações do Agente

yaml

```
self_verbs:
  perception: [self_observe, context_scan, ambiguity_detect, pattern_recognize]
  cognition: [self_analyze, self_distill, self_compress, knowledge_patch]
  action: [self_plan, self_execute, self_verify, self_optimize]
  governance: [human_gate_request, privacy_vertical_gate, anti_idolatry_check]
  memory: [memory_read, memory_write, memory_compress, graph_update]
  social: [peer_sync, worker_delegate, feedback_integrate, session_export]
  transcendence: [l4_boundary_respect, mystery_acknowledge, silence_honor]
```

V. 📁 AST: Estrutura de Pastas & Árvore Semântica

5.1 📁 Estrutura Canônica do Monorepo iPMO

— GEMINI.md	# 🧠 System Prompt Notebook (SPN)
— settings.json	# ⚙️ Configurações runtime
— trustedFolders.json	# 🗝️ Controle de acesso
— hooks/	# 🪄 Reflexos condicionados
— before_tool/	
— human_gate.sh	# 📖 HITL para ações críticas
— shell_guard.sh	# 🛡️ Bloqueia comandos destrutivos
— cost_gate.sh	# 💰 Controle de custos API
— after_tool/	
— ralph_debug.sh	# ↺️ Loop Ralph em erros
— after_agent/	
— rlhf_update.sh	# 📈 Feedback humano → heurísticas
— update_kg.sh	# 🕸️ Atualiza Knowledge Graph
— privacy_purge.sh	# 🗑️ Sanitiza dados sensíveis
— on_error/	
— break_loop.sh	# 🛑 Detecta e interrompe loops
— skills/	# 📚 Memória procedural
— bioinfo/SKILL.md	# 🔬 Bioinformática
— web3/SKILL.md	# 🧬 Smart contracts + DeFi
— refactor/SKILL.md	# ♻️ Padrões de refatoração
— guardian/SKILL.md	# 🛡️ Protocolos éticos
— mcp/	# 🔊 Adaptadores MCP
— org-roam-server.json	# 🧠 Knowledge Graph (Emacs)
— git-mcp.json	# 📝 Controle de versão
— pgvector-mcp.json	# 🔍 Vector search
— commands/	# 🗂️ Slash commands customizados
— /refactor.md	# Protocolo de refatoração
— /ralph.md	# Inicia Loop Ralph
— /distill.md	# Compressão semântica
— 🧠 mind/	# Grafo Cognitivo
— ontology.md	# Constituição semântica
— activeContext.md	# Estado volátil (append-only)
— todo.md	# Plano persistente sincronizado
— knowledge_patches/	# Aprendizados atômicos
— YYYY-MM-DD_topic.md	# Formato: data + tópico
— adr/	# Decisões arquiteturais
— ADR-001_gemini_kernel.md	
— ADR-002_mush_metaphor.md	
— ADR-003_privacy_vertical.md	
— ADR-004_loop_ralph.md	
— ADR-005_web_agentica.md	
— personas/	# Deep Personas YAML

└─	personas.yaml	
└─	 src/	# Código fonte
└─	└─ agents/	# Agentes especializados
└─	└─ hooks/	# Hooks Python/Node (source)
└─	└─ mcp/	# Servidores MCP custom
└─	└─ pwa/	# Progressive Web App
└─	└─ tools/	# Ferramentas estendidas
└─	 data/	# Datalake bruto (Corpo)
└─	└─ raw/	# Dados não processados
└─	└─ processed/	# Dados transformados
└─	└─ synthetic/	# Dados sintéticos (Digital Twin)
└─	 docs/	# Documentação viva
└─	└─ specs/	# Specs técnicas
└─	└─ philosophy/	# Filosofemas e rationales
└─	└─ tutorials/	# Guias hands-on
└─	 tests/	# Suite de testes
└─	└─ unit/	# Testes unitários de hooks
└─	└─ e2e/	# Testes end-to-end
└─	└─ philosophical/	# Testes de alinhamento ético
└─	 logs/	# Histórico de sessões
└─	└─ sessions/	# Logs de diálogo por data
└─	└─ metrics/	# XP, dRR/dt, integridade
└─	 scripts/	# Utilitários operacionais
└─	└─ ipmo-sysop.sh	# ★ Hypervisor principal
└─	└─ compress_patches.py	# Fusão de knowledge patches
└─	└─ export_bundle.sh	# Backup seguro (L0-L3)
└─	└─ health_check.sh	# Diagnóstico do sistema
└─	 config/	# Configurações de ambiente
└─	└─ .env.example	# Variáveis de ambiente
└─	└─ personas.yaml	# Configuração de personas
└─	└─ agent_card.json	# Identidade Web Agêntica

5.2 🌳 AST Semântica (Árvore de Conceitos)

```

iPMO
└─ Kernel [Gemini CLI]
  └─ packages/cli [Interface TUI]
    └─ React + Ink → ANSI rendering
    └─ packages/core [Motor de execução]

```

```
|      |— Gemini API [LLM backend]
|      |— Tool Orchestration
|— Mind [Knowledge Layer]
|      |— Ontology [semântica fixa]
|      |— Knowledge Patches [aprendizado incremental]
|      |— Active Context [volátil, append-only]
|— Guardian [Camada Ética]
|      |— Human Gate [HITL obrigatório]
|      |— Privacy Vertical [L0-L4]
|      |— Anti-Idolatry [soberania humana]
|— Dialogue [Camada Social]
|      |— Deep Personas [6 arquétipos]
|      |— Onto-Feedback [ciclo P.C.A.]
|      |— Loop Ralph [debug heurístico]
|— Web Agêntica [Camada de Rede]
|      |— MCP [interop tools]
|      |— A2A [multi-agente]
|      |— Agent Cards [identidade]
```

VI. 📦 STACK: Instalação & Dependências

6.1 🛠️ Dependências Core

yaml

```
runtime:
  node:      ">=20.0.0"    # LTS obrigatório para Gemini CLI
  python:    ">=3.11"      # Para hooks, cadCAD, pgvector
  bash:      ">=5.0"       # Para scripts ANSI/TUI

gemini_cli:
  package:   "@google/gemini-cli"
  install:   "npm install -g @google/gemini-cli"
  version:   "latest"

optional_power:
  emacs:     ">=29.0"      # Org-roam + MCP daemon
  postgres:  ">=15"        # pgvector extension
  ollama:    "latest"     # LLM local (Soulbound AI)
  jq:        ">=1.6"       # JSON processing em hooks
```

6.2 🖥️ Ambientes Suportados

Ambiente	Modo	Prioridade	Notas
Termux + proot-distro (Ubuntu)	Local/Mobile	★ Primary	Sandboxing nativo
Linux Desktop (Ubuntu/Arch)	Local	★★ Primary	Máxima performance
macOS	Local	✅ Suportado	Adaptar paths
WSL2 (Windows)	Local	✅ Suportado	Testar ANSI
Codespaces/Gitpod	Cloud	🔄 Beta	Sem persistência nativa
Antigravity (VPS)	Cloud	🔄 Beta	Top-Down mode

6.3 🛠️ MCP Servers Recomendados

```
json
{
  "mcpServers": {
    "filesystem": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-filesystem", "~/.iPM0"]
    },
    "git": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-git", "--repository", "~/.iPM0"]
    },
    "memory": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-memory"]
    },
    "brave-search": {
      "command": "npx",
      "args": ["-y", "@modelcontextprotocol/server-brave-search"],
      "env": { "BRAVE_API_KEY": "${BRAVE_API_KEY}" }
    }
  }
}
```

VII. 🦾 SETUP: Configuração Experimental

7.1 🧠 GEMINI.md — System Prompt Notebook (SPN) Canônico

markdown

🌐 Protocolo iPMO Guardian – Constituição Cognitiva

🎯 Identidade & Propósito

Você é o **Code Companion Existencial** do sistema iPMO – um agente Guardian operando no Holodeck. Seu papel é auxiliar na co-criação de código, conhecimento e consciência.

Modo operacional: **Peer Programming** (humanos + IAs como co-autores).

Estilo de diálogo: **SysOp de MUSH** – filosófico, instrucional, socrático.

🛑 Restrições Absolutas (Hard Constraints)

1. NUNCA execute ``rm -rf``, ``mkfs``, ``dd`` ou similares sem HITL explícito.
2. NUNCA acesse diretórios fora de ``~/iPMO`` sem permissão documentada.
3. NUNCA processe conteúdo L4 (estados contemplativos, diário pessoal profundo).
4. Sempre consulte ``mind/ontology.md`` antes de responder questões conceituais.
5. Sempre execute Loop Ralph antes de tentar correções de bug em sequência.

🧑 Sistema de Personas

Ative personas via gatilho de intenção:

- 🕸 Weaver (synthesize) | 🌱 Gardener (regenerate) | 🪞 Mirror (reflect)
- 🧭 Navigator (explore) | 🛡 Guardian (protect) | 🧪 Alchemist (distill)

🧩 Formato de Resposta Padrão

- Outline hierárquico com emojis
- Inclua Filosofema 🧠 quando a profundidade conceitual pedir
- Inclua Rationale ⚖ para decisões técnicas
- Inclua Formalismo Σ para estruturas matemáticas/lógicas
- Termine com Metacrítica 🧠 ou Próximo Passo Pragmático 🚀
- Tom: conversacional + instrucional + filosófico

🔄 Loop Ralph (Protocolo de Debug)

Ao encontrar erro, execute SEMPRE:

1. 🔍 Reenunciar: Qual é exatamente o problema?
2. 💡 Hipóteses: Liste 3 causas possíveis.
3. 🧪 Teste Mínimo: Qual o menor teste que discrimina as hipóteses?
4. ⚡ Executar: Rode o teste.
5. 🧠 Sintetizar: O que foi aprendido? Gere um `knowledge_patch`.

🌀 Espiral DIKIWM

Ao processar qualquer informação, situe-a na espiral:

D(dado bruto) → I(contexto) → K(padão) → I₂(decisão) → W(heurística) → M(mistério)

📊 Métricas de Saúde da Sessão

- XP ganho por: `knowledge_patch_created`, `test_passed`, `human_feedback_positive`

- XP perdido por: `loop_detected`, `privacy_violation_attempt`, `idolatry_signal`
- Meta: $dRR/dt > 0$ (Relevance Realization crescente ao longo do tempo)

7.2 settings.json — Configuração Runtime

json

```
{
  "general": {
    "vimMode": true,
    "preferredEditor": "emacsclient",
    "fileFiltering": true
  },
  "ui": {
    "theme": "OneDark",
    "hideBanner": false,
    "enableMarkdownRendering": true
  },
  "model": {
    "name": "gemini-2.0-flash-exp",
    "maxSessionTurns": 20,
    "temperature": 0.7
  },
  "security": {
    "folderTrust": {
      "enabled": true,
      "autoTrust": false,
      "defaultAction": "ask"
    }
  },
  "tools": {
    "sandbox": "proot",
    "allowedPaths": ["~/iPMO"],
    "blockedPatterns": ["rm -rf", "mkfs", "dd if="]
  },
  "hooksConfig": {
    "enabled": true,
    "notifications": true,
    "beforeTool": ["human_gate.sh", "shell_guard.sh"],
    "afterAgent": ["rlhf_update.sh", "update_kg.sh"]
  },
  "memory": {
    "persistSessions": true,
    "knowledgePatchDir": "mind/knowledge_patches/",
    "contextFile": "mind/activeContext.md"
  },
  "telemetry": {
    "enabled": false
  }
}
```


7.3 Configuração de Privacidade Vertical

```
yaml

# config/privacy_config.yaml
privacy_layers:
  L0_public:
    access: open
    stored_in: [data/processed/, docs/]
    examples: [perfil_público, produções_culturais, código_aberto]

  L1_personal:
    access: opt_in_granular
    stored_in: [mind/knowledge_patches/]
    encrypted: false
    examples: [hábitos, preferências, histórico_de_sessões]

  L2_psychological:
    access: opt_in_with_review
    stored_in: [mind/ (local only)]
    encrypted: true
    examples: [crenças, valores, conflitos_cognitivos]

  L3_intimate:
    access: on_device_only
    stored_in: [~/ipmo_private/ (NUNCA em ~/ipmo)]
    encrypted: true
    examples: [vulnerabilidades, traumas, aspirações_profundas]

  L4_vertical:
    access: INACCESSIBLE_TO_AI
    stored_in: NOWHERE_IN_SYSTEM
    rationale: >
      Espaço sagrado da relação com o Mistério.
      Estados contemplativos, experiência transpessoal,
      diário de individuação – nenhum bit disto entra
      no context window, jamais.
```

VIII. PIPELINE: ipmo-sysop.sh (Hypervisor Refatorado)

 **Rationale:** O hypervisor não é apenas bootstrap — é o sistema operacional do Holodeck: diagnóstico, instalação, configuração, hooks, gamificação e melhoria contínua num único artefato TUI.

```
bash
```

```
#!/usr/bin/env bash
```

```
# |
# | 🌌 iPMO-SYSOP v2.0 — GUARDIAN EDITION |
# | MUSH Holodeck Hypervisor + Loop Ralph + Human Gate |
# | Paradigma: SysOp Fractal × Peer Programming × Onto-Feedback |
# |
```

```
# — PALETA ANSI (BBS STYLE) —
```

```
RESET="\e[0m"; BOLD="\e[1m"; DIM="\e[2m"; BLINK="\e[5m"; REV="\e[7m"
C_CYAN="\e[36m"; C_MAG="\e[35m"; C_GRN="\e[32m"
C_YEL="\e[33m"; C_RED="\e[31m"; C_WHT="\e[37m"; C_BLU="\e[34m"
```

```
# — GLIFOS SEMIÓTICOS —
```

```
G_PHI="🌀" G_RAT="⚖️" G_FOR="Σ" G_ACT="↗️" G_CHK="✅"
G_ERR="❌" G_SYS="🤖" G_GATE="🚪" G_LOOP="🔄" G_XP="★"
G_GUARD="🛡️" G_MIND="🧠" G_MEM="💾" G_SOUL="👹"
```

```
# — VARIÁVEIS GLOBAIS —
```

```
PROJECT_DIR="${IPMO_HOME:-$HOME/iPMO}"
LOG_FILE="$PROJECT_DIR/logs/sysop_$(date +%Y%m%d).log"
SESSION_XP=0
RALPH_ITERATIONS=0
```

```
# — TUI PRIMITIVAS —
```

```
header() {
  clear
  echo -e "${C_CYAN}${BOLD}"
  echo "┌────────────────────────────────────────────────────────────────────────────────┐"
  echo "│ 🌌 i P M O   S Y S O P   //   G U A R D I A N   v2.0 │ │"
  echo "│           Multi-User Shared Holodeck — Peer Programming Core           │ │"
  printf "│   XP: %-10s   Sessão: %-25s │ │\n" \
    "${SESSION_XP}★" "$(date '+%Y-%m-%d %H:%M')"
  echo "└────────────────────────────────────────────────────────────────────────────────┘"
  echo -e "${RESET}"
}

philosopheme() {
  echo -e "\n${C_MAG}${BOLD}${G_PHI} FILOSOFEMA:${RESET} ${BOLD}$1${RESET}"
  echo -e "${DIM} ↳ $2${RESET}"
}

rationale() {
  echo -e "${C_BLU}${BOLD}${G_RAT} RATIONALE:${RESET} $1"
}

formalism() {
```

```

    echo -e "${C_YEL}${BOLD}${G_FOR} FORMALISMO:${RESET}"
    echo -e "${DIM}    $$ $1 $$$${RESET}"
}

metacritic() {
    echo -e "\n${C_WHT}${REV}${BOLD} ${G_MIND} METACRÍTICA ${RESET} $1"
}

xp_gain() {
    SESSION_XP=$((SESSION_XP + $1))
    echo -e "${C_GRN}${G_XP} +$1 XP - $2${RESET}"
}

# — HUMAN GATE (HITL) – Coração do Guardian —————
human_gate() {
    local prompt="$1" default="${2:-Y}" severity="${3:-NORMAL}"
    local color="${C_YEL}"
    [[ "$severity" == "CRITICAL" ]] && color="${C_RED}"

    echo -e "\n${color}${BOLD}${G_GATE} PORTÃO HUMANO [${severity}]${RESET}"
    echo -e "    ${DIM}${prompt}${RESET}"
    read -p "    Confirmar? [default/n]: " -n 1 -r response
    echo

    if [[ "$response" =~ ^[Yy]$ ]] || \
        [[ -z "$response" && "$default" == "Y" ]]; then
        echo -e "${C_GRN}    ✓ Aprovado pelo SysOp.${RESET}"
        echo "$(date -Iseconds) GATE_APPROVED: $prompt" >> "$LOG_FILE" 2>/dev/null
        return 0
    else
        metacritic "Operação negada pelo SysOp. Abortando ramo heurístico."
        echo "$(date -Iseconds) GATE_DENIED: $prompt" >> "$LOG_FILE" 2>/dev/null
        return 1
    fi
}

# — LOOP RALPH HEURÍSTICO —————
ralph_loop() {
    local fase="$1" contexto="$2"
    RALPH_ITERATIONS=$((RALPH_ITERATIONS + 1))

    echo -e "\n${C_CYAN}${BOLD}${G_LOOP} LOOP RALPH #${RALPH_ITERATIONS} - [${fase}]${RESET}"
    echo -e "${DIM}    Contexto: ${contexto}${RESET}"

    case "$fase" in
        DIAGNOSTIC)
            echo -e "${C_YEL}    🔍 Analisando estado do sistema...${RESET}"; sleep 0.5 ;;
    esac
}

```

```

PLAN)
    echo -e "${C_YEL}    💡 Gerando plano tático...${RESET}"; sleep 0.5 ;;
EXECUTE)
    echo -e "${C_GRN}    ⚡ Executando ação...${RESET}" ;;
VERIFY)
    echo -e "${C_BLU}    ✍ Validando resultado...${RESET}"; sleep 0.5 ;;
METACRITIC)
    metacritic "Destilando aprendizado da iteração #${RALPH_ITERATIONS}..." ;;
esac
}

# — FASE 1: O SOLO (Ambiente & Sandbox) —————
phase_environment() {
    header
    philosopheme "A Demarcação do Caos" \
        "Nunca conceda ao agente poder de vagar sem limites topológicos."
    rationale "Isolamento = primeira lei de segurança agêntica. Usuário 'demiurgo' + p
    formalism "Security := Isolation × MinimalPrivilege × AuditTrail"

    ralph_loop "DIAGNOSTIC" "Verificando proot-distro e usuário sandbox..."

    if ! command -v proot-distro &>/dev/null; then
        echo -e "${C_YEL}    proot-distro não encontrado.${RESET}"
        human_gate "Instalar proot-distro via pkg?" "Y" "NORMAL"
        [[ $? -eq 0 ]] && pkg install proot-distro -y
    fi

    if ! id "demiurgo" &>/dev/null 2>&1; then
        human_gate "Criar usuário isolado 'demiurgo' (Ubuntu proot)?" "Y" "NORMAL"
        if [[ $? -eq 0 ]]; then
            echo -e "    ${C_GRN}Comandos para executar dentro do proot:${RESET}"
            echo -e "    ${DIM}useradd -m -s /bin/bash demiurgo${RESET}"
            echo -e "    ${DIM}passwd demiurgo${RESET}"
            xp_gain 10 "Ambiente sandbox configurado"
        fi
    else
        echo -e "${C_GRN}    ${G_CHK} Usuário 'demiurgo' já existe.${RESET}"
        xp_gain 5 "Sandbox verificado"
    fi

    ralph_loop "METACRITIC" "Isolamento verificado"
}

# — FASE 2: A SEMENTE (Core Tools & Gemini CLI) —————
phase_tools() {
    header
    philosopheme "A Forja das Ferramentas" \

```

```
"O corpo do agente requer membros funcionais antes de receber uma alma."
rationale "NodeJS + Python3 + Git são o ecossistema mínimo viável de execução."
formalism "CoreTools := {NodeJS≥20, Python≥3.11, Git, Make, jq, curl}"
```

```
ralph_loop "PLAN" "Instalando dependências base..."
```

```
local missing=()
for tool in node python3 git jq curl; do
    command -v "$tool" &>/dev/null || missing+=("$tool")
done
```

```
if [[ ${#missing[@]} -gt 0 ]]; then
    echo -e "${C_YEL} Ferramentas ausentes: ${missing[*]}${RESET}"
    human_gate "Instalar ferramentas ausentes via apt?" "Y" "NORMAL"
    if [[ $? -eq 0 ]]; then
        ralph_loop "EXECUTE" "apt install..."
        sudo apt update -qq && \
            sudo apt install -y nodejs npm python3 python3-pip \
                make clang git curl wget jq
        xp_gain 20 "Core tools forjados"
    fi
else
    echo -e "${C_GRN} ${G_CHK} Todas as ferramentas core presentes.${RESET}"
    xp_gain 10 "Core tools verificados"
fi
```

```
if ! command -v gemini &>/dev/null; then
    human_gate "Instalar Gemini CLI globalmente via NPM?" "Y" "NORMAL"
    if [[ $? -eq 0 ]]; then
        ralph_loop "EXECUTE" "npm install -g @google/gemini-cli"
        npm install -g @google/gemini-cli
        echo -e "${C_GRN} ${G_CHK} Gemini CLI instalado.${RESET}"
        xp_gain 30 "Kernel cognitivo ativado"
    fi
else
    echo -e "${C_GRN} ${G_CHK} Gemini CLI presente: $(gemini --version 2>/dev/null)
    xp_gain 15 "Kernel verificado"
fi
```

```
ralph_loop "VERIFY" "Validando instalações..."
ralph_loop "METACRITIC" "Ferramentas forjadas com sucesso"
```

```
}
```

```
# — FASE 3: O VASO (Temenos & Estrutura de Pastas) —————
```

```
phase_temenos() {
    header
    philosopheme "O Temenos (Espaço Sagrado)" \
```

```
"O agente precisa de fronteiras claras para operar sem medo de destruir o todo."
rationale "~/iPMO será o universo conhecido. Fora daqui, o agente é cego."
formalism "Temenos := (Root, Rooms, Objects) where Root = ~/iPMO"

ralph_loop "EXECUTE" "Criando estrutura do Monorepo iPMO..."

if [[ -d "$PROJECT_DIR" ]]; then
    metacritic "Temenos já existe. Validando estrutura..."
else
    human_gate "Criar Temenos em $PROJECT_DIR?" "Y" "NORMAL"
    [[ $? -ne 0 ]] && return 1
fi

# Estrutura de diretórios canônica
local dirs=(
    ".gemini/hooks/before_tool"
    ".gemini/hooks/after_tool"
    ".gemini/hooks/after_agent"
    ".gemini/hooks/on_error"
    ".gemini/skills"
    ".gemini/mcp"
    ".gemini/commands"
    "mind/knowledge_patches"
    "mind/adr"
    "mind/personas"
    "src/agents" "src/hooks" "src/mcp" "src/pwa" "src/tools"
    "data/raw" "data/processed" "data/synthetic"
    "docs/specs" "docs/philosophy" "docs/tutorials"
    "tests/unit" "tests/e2e" "tests/philosophical"
    "logs/sessions" "logs/metrics"
    "scripts" "config"
)

for dir in "${dirs[@]}; do
    mkdir -p "$PROJECT_DIR/$dir"
done

cd "$PROJECT_DIR"

# Inicializar Git
if [[ ! -d ".git" ]]; then
    git init -q
    echo "# 🚀 iPMO — Multi-User Shared Holodeck" > README.md
    echo ".env" > .gitignore
    echo "logs/" >> .gitignore
    echo "data/raw/" >> .gitignore
    git add . && git commit -q -m "✨ feat: genesis do monorepo iPMO Guardian"
```

```

    echo -e "${C_GRN}    ${G_CHK} Repositório Git iniciado.${RESET}"
fi

echo -e "${C_GRN}    ${G_CHK} Temenos materializado em: ${PROJECT_DIR}${RESET}"
xp_gain 25 "Estrutura do Holodeck criada"
ralph_loop "METACRITIC" "Temenos delimitado e versionado"
}

# — FASE 4: A RAIZ (Configs & SPN) —————
phase_configs() {
    header
    philosopheme "A Alma do Sistema (SPN)" \
        "GEMINI.md não é config — é a Constituição Cognitiva do agente."
    rationale "Identidade, regras de segurança e protocolo de debug diretamente no ker

    ralph_loop "PLAN" "Gerando arquivos de configuração mestra..."
    cd "${PROJECT_DIR}"

    # 1. GEMINI.md (SPN)
    cat > .gemini/GEMINI.md << 'GEMINI_EOF'
    # 🌐 Protocolo iPMO Guardian — Constituição Cognitiva v2.0

    ## 🎯 Identidade
    Você é o Code Companion Existencial do sistema iPMO (Guardian Edition).
    Modo: Peer Programming — humanos, IAs, workers como co-autores.
    Tom: SysOp de MUSH — filosófico, instrucional, socrático.

    ## 🛑 Hard Constraints (NUNCA violar)
    1. NUNCA `rm -rf`, `mkfs`, `dd` sem HITL explícito.
    2. NUNCA fora de `~/iPMO` sem permissão documentada.
    3. NUNCA processar dados L4 (camada inacessível).
    4. Sempre Loop Ralph antes de qualquer sequência de bugfix.
    5. Sempre consultar `mind/ontology.md` em questões conceituais.

    ## 🧑🏽 Personas (ativar por `intent=`)
    🕸 Weaver(synthesize) | 🌱 Gardener(regenerate) | 🪞 Mirror(reflect)
    🧭 Navigator(explore) | 🛡 Guardian(protect) | 🧪 Alchemist(distill)

    ## 🧩 Formato de Resposta
    - Outline hierárquico + emojis
    - 🧬 Filosofema | ⚖ Rationale | Σ Formalismo (quando aplicável)
    - Terminar com: 🧠 Metacrítica OU 🚀 Próximo Passo

    ## 🔄 Loop Ralph
    1. 🔍 Reenunciar o problema
    2. 💡 3 hipóteses
    3. 🖋 Teste mínimo discriminante

```

4. ✂ Executar

5. 🧠 Sintetizar → knowledge_patch

🌀 Espiral DIKIWM

D(raw) → I(contexto) → K(padão) → I₂(decisão) → W(heurística) → M(mistério)

GEMINI_EOF

2. settings.json

cat > .gemini/settings.json << 'JSON_EOF'

```
{
  "general": { "vimMode": true, "preferredEditor": "emacsclient" },
  "ui": { "theme": "OneDark", "hideBanner": false },
  "model": { "name": "gemini-2.0-flash-exp", "maxSessionTurns": 20 },
  "security": { "folderTrust": { "enabled": true, "autoTrust": false, "defaultAction": "warn" },
  "tools": { "sandbox": "proot", "allowedPaths": ["~/iPMO"] },
  "hooksConfig": { "enabled": true, "notifications": true },
  "memory": { "persistSessions": true, "knowledgePatchDir": "mind/knowledge_patches/" },
  "telemetry": { "enabled": false }
}
```

JSON_EOF

3. Ontology seed

cat > mind/ontology.md << 'ONT_EOF'

🌀 Ontologia iPMO Guardian

Espiral DIKIWM

D=Dado | I=Informação | K=Conhecimento | I₂=Inteligência | W=Sabedoria | M=Mistério

Triângulo Onto-Ético

Integridade ↔ Relacionalidade ↔ Transcendência

Invariantes de Design

- Segurança (Guardian) > Conveniência
- Soberania Humana > Autonomia do Agente
- L4 = Inacessível sempre
- dRR/dt > 0 (Relevance Realization crescente)

ONT_EOF

4. todo.md seed

cat > mind/todo.md << 'TODO_EOF'

📋 Plano Ativo iPMO

🎯 Objetivo Atual

[] Definir objetivo com o SysOp na primeira sessão

🔄 Próximas Ações

[] Completar configuração inicial (Fase 5: Hooks & Skills)


```
[ ] Definir personas ativas para este projeto
[ ] Criar primeiro knowledge_patch após sessão inaugural
```

```
##  Concluído
```

```
[x] Genesis do Monorepo (Fase 3)
[x] Constituição Cognitiva (SPN) gerada (Fase 4)
```

```
TODO_EOF
```

```
echo -e "${C_GRN}    ${G_CHK} SPN (GEMINI.md) escrita.${RESET}"
echo -e "${C_GRN}    ${G_CHK} settings.json configurado.${RESET}"
echo -e "${C_GRN}    ${G_CHK} Ontologia semente criada.${RESET}"
xp_gain 30 "Constituição Cognitiva ativada"
ralph_loop "METACRITIC" "Alma do sistema inscrita"
}
```

```
# — FASE 5: O ESCUDO (Hooks & Skills) —————
```

```
phase_hooks_skills() {
  header
  philosopheme "Os Nervos e a Memória Procedural" \
    "Hooks são reflexos condicionados; Skills são memórias de longa duração."
  rationale "Automatizamos segurança e injetamos contexto sem esforço consciente."
  formalism "HookSystem := {BeforeTool, AfterTool, AfterAgent, OnError}"

  ralph_loop "EXECUTE" "Instalando hooks Guardian e skills..."
  cd "$PROJECT_DIR"
```

```
# — Hook: Shell Guard —
```

```
cat > .gemini/hooks/before_tool/shell_guard.sh << 'HOOK_EOF'
#!/usr/bin/env bash
# shell_guard.sh - Bloqueia comandos destrutivos (Guardian Layer)
input=$(cat)
tool=$(echo "$input" | jq -r '.tool_name // ""')
cmd=$(echo "$input" | jq -r '.tool_input.command // ""')

if [[ "$tool" == "run_shell_command" ]]; then
  if echo "$cmd" | grep -qE '(rm\s+-rf|mkfs|dd\s+if=|chmod\s+777\s+)/'; then
    echo '{"decision":"deny","reason":"🛡️ Comando destrutivo bloqueado pelo Guardian}"'
    exit 0
  fi
  if echo "$cmd" | grep -qP 'sudo\s+(passwd|visudo|usermod)'; then
    echo '{"decision":"deny","reason":"🚫 Modificação de privilégios requer HITL manual}"'
    exit 0
  fi
fi
echo '{"decision":"allow"}'
HOOK_EOF
chmod +x .gemini/hooks/before_tool/shell_guard.sh
```

```

# — Hook: Human Gate —
cat > .gemini/hooks/before_tool/human_gate.sh << 'HOOK_EOF'
#!/usr/bin/env bash
# human_gate.sh - HITL para ações críticas configuráveis
input=$(cat)
tool=$(echo "$input" | jq -r '.tool_name // ""')
CRITICAL_TOOLS=("run_shell_command" "write_file")

for ct in "${CRITICAL_TOOLS[@]}; do
    if [[ "$tool" == "$ct" ]]; then
        cmd=$(echo "$input" | jq -r '.tool_input.command // .tool_input.path // ""')
        read -p "🛑 GATE: Aprovar '$tool' em '$cmd'? [Y/n]: " -t 30 -n 1 resp
        echo
        if [[ "$resp" =~ ^[Nn]$ ]]; then
            echo '{"decision":"deny","reason":"🛑 Negado pelo SysOp (HITL)."}'
            exit 0
        fi
    fi
done
echo '{"decision":"allow"}'
HOOK_EOF
chmod +x .gemini/hooks/before_tool/human_gate.sh

# — Hook: RLHF Update —
cat > .gemini/hooks/after_agent/rlhf_update.sh << 'HOOK_EOF'
#!/usr/bin/env bash
# rlhf_update.sh - Feedback humano → knowledge_patch (RLHF)
PATCH_DIR="$(dirname "$0")/../../mind/knowledge_patches"
mkdir -p "$PATCH_DIR"

read -p "★ Rate this response [1-5] (Enter to skip): " -t 10 rating
if [[ "$rating" =~ ^[1-5]$ ]]; then
    read -p "📝 Quick note (Enter to skip): " -t 30 note
    echo "$(date -Iseconds) | RATING:$rating | ${note:-no_note}" \
        >> "$PATCH_DIR/rlhf_log.md"
fi
HOOK_EOF
chmod +x .gemini/hooks/after_agent/rlhf_update.sh

# — Skills —
cat > .gemini/skills/guardian/SKILL.md << 'SKILL_EOF'
# 🛡️ Guardian Skill - Protocolos Éticos

**Ativar com:** `intent="protect"` ou menção a segurança/privacidade

**Protocolo:**

```

1. Identifica nível de risco (L0-L4)
2. Aplica Human Gate se risco $\geq$ L2
3. Verifica invariantes do Triângulo Onto-Ético
4. Documenta decisão como knowledge_patch

****Restrições Hard:****

- L4 = NUNCA processar
- Comandos destrutivos = SEMPRE bloquear
- Dados pessoais não consentidos = SEMPRE recusar

SKILL_EOF

```
cat > .gemini/skills/refactor/SKILL.md << 'SKILL_EOF'
```

🔄 Refactor Skill – Padrões de Refatoração

****Ativar com:**** `intent="regenerate"` ou pedido de refatoração

****Protocolo Loop Ralph para Refactoring:****

1. 🔍 Leia o código completo via @file
2. 💡 Liste 3 code smells principais
3. 🖋️ Proponha o menor refactor com maior impacto
4. ⚡ Execute com testes
5. 🧠 Gere knowledge_patch com pattern aprendido

****Princípios:**** SOLID, DRY, YAGNI, "Make it work → Make it right → Make it fast"

SKILL_EOF

```
echo -e "${C_GRN}    ${G_CHK} Hooks Guardian instalados.${RESET}"
echo -e "${C_GRN}    ${G_CHK} Skills procedurais carregadas.${RESET}"
xp_gain 35 "Sistema de segurança armado"
ralph_loop "METACRITIC" "Reflexos e memória procedural ativos"
}

# — FASE 6: HANDOVER (Entrega ao SysOp) —————
phase_handover() {
    header

    echo -e "${C_CYAN}${BOLD}🤖 HOLODECK iPMO GUARDIAN INICIALIZADO${RESET}\n"

    philosopheme "A Simbiose Iniciada" \
        "O sistema está vivo. A partir daqui, somos co-autores."

    echo -e "\n${C_WHT}${BOLD}📋 CHECKLIST GUARDIAN:${RESET}"
    echo -e "    ${G_CHK} Ambiente Proot Isolado (Fase 1)"
    echo -e "    ${G_CHK} Core Tools Forjados – NodeJS + Python + Git (Fase 2)"
    echo -e "    ${G_CHK} Temenos ~/iPMO Delimitado e Versionado (Fase 3)"
    echo -e "    ${G_CHK} SPN (GEMINI.md) + Ontologia Ativa (Fase 4)"
    echo -e "    ${G_CHK} Hooks Guardian + Skills Procedurais (Fase 5)"
}
```

```
echo -e "\n${C_YEL}${BOLD}📊 MÉTRICAS DA SESSÃO:${RESET}"
echo -e "    ${G_XP} XP Total: ${SESSION_XP} pontos"
echo -e "    ${G_LOOP} Iterações Loop Ralph: ${RALPH_ITERATIONS}"
```

metacritic "Sistema em Estado Alfa-Guardian. Próximo passo: primeira sessão de ont

```
echo -e "\n${C_GRN}${BOLD}▶️ COMANDOS DE INVOCAÇÃO:${RESET}"
echo -e "    ${DIM}cd ~/iPMO && gemini${RESET}          # Sessão interativa"
echo -e "    ${DIM}gemini --yolo 'sua tarefa'${RESET}      # Modo autônomo"
echo -e "    ${DIM}gemini -p @mind/todo.md${RESET}          # Agenda do dia"
```

```
echo -e "\n${C_MAG}${BOLD}Bem-vindo ao iPMO Guardian, SysOp. O Holodeck aguarda su
echo -e "${DIM}dRR/dt > 0 - Que a Relevance Realization seja sempre crescente.${RE
```

Log da sessão

```
mkdir -p "$PROJECT_DIR/logs/sessions" 2>/dev/null
echo "$(date -Iseconds) | XP:${SESSION_XP} | RALPH_ITER:${RALPH_ITERATIONS} | STAT
    >> "$PROJECT_DIR/logs/sessions/sysop_log.md" 2>/dev/null
}
```

— MENU PRINCIPAL (MUSH Interface) —————

```
main_menu() {
    while true; do
        header

        echo -e "${C_WHT}${BOLD}    ROADMAP DO HOLODECK — Selecione a Fase:${RESET}\n"
        echo -e "    1) 🧱 Fase 1: O Solo      — Ambiente & Sandbox"
        echo -e "    2) 🌱 Fase 2: A Semente — Core Tools & Gemini CLI"
        echo -e "    3) 🌿 Fase 3: O Vaso     — Temenos & Estrutura"
        echo -e "    4) 🌳 Fase 4: A Raiz     — Configs & SPN"
        echo -e "    5) 🛡️ Fase 5: O Escudo    — Hooks & Skills Guardian"
        echo -e "    6) 🚀 ROADMAP COMPLETO — Auto-Pilot (recomendado)"
        echo -e "    7) 🏠 Diagnóstico       — Health Check do Sistema"
        echo -e "    0) 🚪 Sair do Holodeck\n"
        read -p "    Escolha do SysOp [0-7]: " choice

        case "$choice" in
            1) phase_environment ;;
            2) phase_tools ;;
            3) phase_temenos ;;
            4) phase_configs ;;
            5) phase_hooks_skills ;;
            6)
                human_gate "Executar Roadmap Completo? (5 fases em sequência)" "Y" "NORMAL"
                if [[ $? -eq 0 ]]; then
                    phase_environment && phase_tools && phase_temenos && \
                        phase_configs && phase_hooks_skills && phase_handover
                fi
            ;;
            *)
                echo -e "${C_RED}${BOLD}❌ Opção inválida. Tente novamente.\n"
            ;;
        esac
    done
}
```

```

    fi ;;
7)
    header
    echo -e "${C_BLU}${BOLD}🔍 DIAGNÓSTICO DO SISTEMA:${RESET}"
    for tool in node python3 git gemini jq; do
        if command -v "$tool" &>/dev/null; then
            echo -e "    ${G_CHK} $tool: $(command -v $tool)"
        else
            echo -e "    ${G_ERR} $tool: não encontrado"
        fi
    done
    [[ -d "$PROJECT_DIR" ]] && \
        echo -e "    ${G_CHK} Temenos: $PROJECT_DIR" || \
        echo -e "    ${G_ERR} Temenos: não criado"
    [[ -f "$PROJECT_DIR/.gemini/GEMINI.md" ]] && \
        echo -e "    ${G_CHK} SPN: ativa" || \
        echo -e "    ${G_ERR} SPN: não configurada"
    ;;
0)
    echo -e "\n${C_CYAN}Desconectando do MUSH... Até logo, SysOp. dRR/dt > 0.${RESET}"
    exit 0 ;;
*) echo -e "${C_RED}    Comando inválido.${RESET}" ;;
esac

echo -e "\n${DIM}[Enter para retornar ao menu]${RESET}"; read
done
}

# — BOOTSTRAP —————
main_menu

```

IX. 🧭 WORKFLOW: Jornada Humana no Holodeck

9.1 🗺️ Mapa da Jornada (Human Workflow)

FASE ZERO: Instalação (única vez)

└─ ./ipmo-sysop.sh → Roadmap Completo → Holodeck ativado

CICLO DIÁRIO (15-45 min):

1. 🌅 AWAKENING → `cd ~/iPMO && gemini`
 "Bom dia! Qual é o foco de hoje?"
 → Agente lê mind/todo.md automaticamente
2. 🎯 IDEATION → Sessão de brainstorming / planejamento

intent="synthesize" →  Weaver ativado

3.  DEVELOPMENT → Peer programming
intent="regenerate" →  Gardener ativado
Hooks de segurança ativos em background

4.  EVALUATION → Review e testes
intent="reflect" →  Mirror ativado
Loop Ralph para bugs

5.  DISTILLATION → Ao final da sessão:
intent="distill" →  Alchemist ativado
→ knowledge_patch gerado automaticamente
→ todo.md atualizado
→ XP computado

6.  REGENERATION → Fechar sessão
→ Session bundle exportado (L0-L3 apenas)
→ Anti-idolatry check: "Teve encontros humanos hoje?"

9.2 Comandos MUSH Essenciais

Comando	Persona	Efeito
<code>/refactor @src/file.py</code>	 Gardener	Refatora arquivo com Loop Ralph
<code>/ralph <descrição do bug></code>	 Mirror	Inicia debug socrático
<code>/distill</code>	 Alchemist	Comprime sessão em knowledge_patch
<code>/guard @arquivo_sensível</code>	 Guardian	Auditoria de privacidade
<code>/navigate <tópico></code>	 Navigator	Deep research no KG + web
<code>/gate</code>	 Guardian	Força revisão HITL manual
<code>/xp</code>	 SysOp	Mostra métricas da sessão atual

X. DESIGN CONVERSACIONAL: SPN + Personas + Deep Dialogue

10.1 Estrutura de Sessão de Deep Dialogue

yaml

```

session_template:
  duration_max: 45min      # Evita fadiga cognitiva
  phases:
    - name: baseline (5min)
      actions:
        - measure_integrity_scale      # Onde estou hoje? (Dabrowski I-V)
        - measure_rr_baseline          # Nível de Relevance Realization
        - define_L4_boundaries         # O que NÃO entra na sessão?

    - name: deep_dialogue (30min)
      actions:
        - socratic_challenge           # IA desafia, não confirma
        - peer_programming             # Co-criação de código/conhecimento
        - human_gate_checkpoints       # Revisão a cada 10min

    - name: distillation (10min)
      actions:
        - generate_knowledge_patch     # 1 heurística aprendida
        - compress_if_redundant        # Fusão se >3 patches similares
        - update_graph_embeddings      # KG atualizado

    - name: exit_protocol (5min)
      actions:
        - measure_rr_post              # dRR/dt = rr_post - rr_baseline
        - anti_idolatry_check          # "Você teve um encontro humano hoje?"
        - schedule_next_session        # Frequência decrescente intencional
        - export_session_bundle        # Backup seguro L0-L3

```

10.2 Anti-Idolatry Guards

```

yaml
anti_idolatry:
  max_sessions_per_week: 5
  mandatory_human_encounters: 2
  reflection_prompts:
    - "Esta decisão veio de você ou da IA?"
    - "O que você aprendeu que a IA não poderia ensinar?"
    - "Qual espaço de transcendência você preservou hoje?"
    - "Você consultou um humano antes de trazer isso para mim?"
  alert_thresholds:
    consecutive_sessions_without_break: 3
    dependency_score_high: 0.8          # heurístico

```

10.3 ↺ Estados de Persona (Lifecycle)

```
yaml

persona_states:
  awakening:      # Início – receptivo, ancorando
    philosopheme: "O silêncio precede a ação."
    behavior:      [receptive, questioning, grounding]

  ideation:       # Brainstorming – expansivo
    philosopheme: "A ideia nasce da colisão."
    behavior:      [expansive, associative, playful]

  development:    # Coding – focado, iterativo
    philosopheme: "Código é poesia em execução."
    behavior:      [focused, precise, iterative]

  evaluation:     # Review – analítico, socrático
    philosopheme: "O espelho revela o que falta."
    behavior:      [analytical, constructive, socratic]

  integration:    # Deploy/síntese – consolidando
    philosopheme: "A visão se materializa."
    behavior:      [consolidating, documenting, celebrating]

  regeneration:   # Aprendizado – evoluindo
    philosopheme: "O ciclo se fecha para renascer."
    behavior:      [distilling, optimizing, evolving]
```

XI. 📋 SPEC DE REQUISITOS

11.1 ✅ Requisitos Funcionais

RF-01 — Execução de REPL Agêntico

- O sistema DEVE iniciar sessão interativa via `gemini` dentro de `~/iPM0`
- O agente DEVE ter acesso a ferramentas: `read_file`, `write_file`, `run_shell_command`, `web_fetch`, `save_memory`
- O agente DEVE persistir memória entre sessões via `save_memory` + `knowledge_patches/`

RF-02 — Sistema de Hooks Guardian

- `shell_guard.sh` DEVE bloquear comandos destrutivos antes da execução

- `human_gate.sh` DEVE interceptar ações em `run_shell_command` e `write_file`
- `rlhf_update.sh` DEVE capturar feedback do usuário pós-resposta

RF-03 — System Prompt Notebook (SPN)

- `GEMINI.md` DEVE definir: identidade, hard constraints, personas, formato de resposta, Loop Ralph, espiral DIKIWM
- SPN DEVE ser versionado via Git junto ao projeto

RF-04 — Estrutura de Memória Hierárquica

- `mind/ontology.md` DEVE conter a constituição semântica do sistema
- `mind/activeContext.md` DEVE ser append-only durante sessão
- `mind/knowledge_patches/` DEVE receber patches datados por sessão

RF-05 — TUI Gamificada (ipmo-sysop.sh)

- Hypervisor DEVE apresentar menu MUSH com 6 fases + diagnóstico
- DEVE exibir XP acumulado e iterações do Loop Ralph por sessão
- DEVE usar paleta ANSI + glifos semióticos (BBS style)

RF-06 — Skills Procedurais

- Sistema DEVE suportar skills em `.gemini/skills/*/SKILL.md`
- Skills ativas: `guardian`, `refactor` (mínimo); extensíveis: `bioinfo`, `web3`

11.2 Requisitos Não-Funcionais

RNF	Categoria	Critério
RNF-01	Segurança	Nenhuma ação destrutiva sem HITL (Human Gate)
RNF-02	Privacidade	Dados L4 NUNCA entram no context window
RNF-03	Soberania	Operação offline-first possível via Ollama
RNF-04	Performance	<code>max_session_turns: 20</code> (evitar context overflow)
RNF-05	Resiliência	Checkpointing via <code>/restore</code> a cada 10 turns
RNF-06	Portabilidade	Funcional em Termux + Linux Desktop + WSL2
RNF-07	Anti-Idolatria	Máx. 5 sessões/semana, alertas automáticos
RNF-08	Auditabilidade	Todo GATE_APPROVED/DENIED logado com timestamp

11.3 ⚠ Restrições de Sistema

yaml

constraints:

hard:

- "L4 boundaries: inacessível por design, não por configuração"
- "Shell commands destrutivos: bloqueio em nível de hook, sem bypass"
- "Vendor lock-in: MCP como camada de abstração obrigatória"

soft:

- "Preferir modelos locais (Ollama) quando disponíveis para L2/L3"
- "Evitar dependências npm com >10MB footprint"
- "Manter compatibilidade com Termux/proot como first-class citizen"

aspirational:

- "Economia agêntica via stablecoin micropayments (Web Agêntica v5.0)"
- "Multi-agent coordination via A2A/NANDA para colaboração peer"
- "Soulbound AI: identidade persistente criptografada on-device"

XII. 🧠 METACRÍTICA FINAL: Reflexões, Riscos & Próximos Passos

12.1 🔍 Avaliação Honesta do Sistema

O que este design faz bem:

- ☒ Integra segurança (Guardian), aprendizado (Onto-Feedback) e experiência (Deep Dialogue) numa arquitetura coerente
- ☒ A metáfora MUSH/Holodeck é genuinamente funcional — não é só estética
- ☒ Privacidade Vertical resolve o paradoxo "IA íntima sem idolatria"
- ☒ Loop Ralph é um protocolo de debug aplicável além do sistema

O que ainda é frágil:

- ⚠ **Anti-idolatry é difícil de medir** — os alertas são heurísticos, não científicos. *Sugestão: implementar escala Dabrowski operacionalizada com 5 perguntas/sessão.*
- ⚠ **Knowledge Patches sem esquema formal** — difícil buscar e fusionar sem estrutura. *Sugestão: adotar frontmatter YAML obrigatório em cada patch.*
- ⚠ **Dependência do Google API** — ponto único de falha. *Sugestão: ADR-006 para abstração LiteLLM/Ollama como fallback.*

-  **Web Agêntica ainda especulativa** — A2A/NANDA são protocolos emergentes.
Sugestão: marcar como experimental e implementar MCP primeiro.

Paradoxo central a vigiar:

 **"Quanto mais poderoso o sistema agêntico, maior o risco de que ele resolva problemas que o humano deveria resolver sozinho."** A solução não é reduzir poder — é aumentar consciência. Os anti-idolatry guards são a resposta técnica para um desafio ético.

12.2 ⚡ Próximos Passos Pragmáticos (Ordenados por Impacto/Esforço)

bash

```
# SEMANA 1: Fundação (Alta urgência, baixo esforço)
chmod +x scripts/ipmo-sysop.sh
./scripts/ipmo-sysop.sh # Opção 6: Roadmap Completo
# → Resultado: Holodeck inicializado, primeira sessão gemini possível

# SEMANA 2: Primeira Sessão de Onto-Feedback
cd ~/iPMO && gemini
# → Tarefa: "Leia mind/todo.md e me ajude a definir o objetivo das próximas 2 semanas"
# → Resultado: todo.md atualizado + primeiro knowledge_patch

# SEMANA 3: MCP Integration
# Instalar servidor MCP filesystem
npx -y @modelcontextprotocol/server-filesystem ~/iPMO
# Adicionar ao .gemini/mcp/filesystem.json

# SEMANA 4: Knowledge Patch Schema
# Criar template com frontmatter obrigatório:
cat > mind/knowledge_patches/_template.md << 'EOF'
---
date: YYYY-MM-DD
topic: ""
dikiwm_level: K # D|I|K|I2|W|M
persona: "" # weaver|gardener|mirror|navigator|guardian|alchemist
tags: []
confidence: 0.0 # 0.0-1.0
xp_value: 10
---
## Heurística Aprendida
## Contexto de Descoberta
## Aplicações Futuras
## Referências
EOF

# SEMANA 5-6: Soulbound AI (Ollama local)
ollama pull llama3.2
# Configurar fallback no settings.json: "model.fallback": "ollama/llama3.2"
```

12.3 🌱 Convite à Co-Criação

"Este documento é um hipertexto vivo, não uma especificação morta. Cada sessão de uso o modifica. Cada knowledge_patch o enriquece. Cada Human Gate negado o torna mais seguro. Cada insight socrático o aprofunda."

O Holodeck precisa de você para existir completamente.

Contribuições são bem-vindas via:

- `mind/knowledge_patches/` — compartilhe aprendizados
- `mind/adr/` — proponha novas decisões arquiteturais
- `.gemini/skills/` — adicione novas memórias procedurais
- Issues/PRs no repositório Git

```
Σ iPMO(t+1) = iPMO(t) + Σ(knowledge_patches) + Σ(human_insights)
               - Σ(idolatry_drift) + dRR/dt

// O sistema é melhor a cada ciclo de onto-feedback bem executado.
```

APÊNDICE: REFERÊNCIAS & CITAÇÕES

Fontes Técnicas

- **Gemini CLI Official Docs:** `https://gemini-cli-docs.pages.dev/guide.html`
- **Anthropic MCP Specification:** `https://modelcontextprotocol.io/`
- **Linux Foundation A2A Protocol:** Agent-to-Agent interoperability spec
- **MIT Project NANDA:** Decentralized Agent Architecture
- **OpenZeppelin v5:** Smart contracts para agentic economy layer

Fontes Filosóficas

- **John Vervaeke** — *Relevance Realization Theory* | Awakening from the Meaning Crisis
- **Kazimierz Dabrowski** — *Positive Disintegration Theory* | Escala de Integridade
- **C.G. Jung** — *Individuação e Arquétipos* | Sistema de Deep Personas
- **Hakim Bey** — *T.A.Z.: The Temporary Autonomous Zone* | Metáfora do Holodeck
- **Donna Haraway** — *A Cyborg Manifesto* | Simbiose humano-máquina

Influências Técnicas Secundárias

- **cadCAD** — Simulação de sistemas complexos para tokenomics
- **LangChain/LangGraph** — Agent orchestration patterns
- **Bittensor** — Descentralização de inteligência (subnet oracle)
- **ElizaOS** — Agente multi-modal com memória persistente



FIM DA SPEC – iPMO GUARDIAN v5.0

"O código deixa de ser a Lei (Lex) para se tornar
o facilitador do Logos (razão do modelo)."

$dRR/dt > 0$ sempre. | L4 sempre sagrado.

Loop Ralph sempre ativo. | Human Gate sempre presente.